

Task 1: Supervised Learning

1. Introduction & Aims

Supervised learning is a category of machine learning that uses labeled datasets to train algorithms to predict outcomes and recognize patterns.

Supervised learning contains two types:

- Classification: The output is a discrete value (e.g. gas on/off). The model assigns each input to any of the finite set of classes.
- Regression: The output is a continuous value (e.g. gas volume in m³). The model predicts the value.

The aim of this task is to compare the performance (accuracy and run time) of two or more machine learning algorithms covered in the lectures. In order to achieve this aim, the following objectives will be to:

- train the regression models using the training dataset to predict the hourly energy consumption of homes using input features
- evaluate the accuracy and run time of regression algorithms applied using the testing and validation datasets
- establish the best performing algorithm for this data

Using supervised learning in the context of Smart Buildings, helps us to make predictions on energy consumption from various data associated with living conditions, like ideal temperature humidity, external weather conditions, etc to determine how much energy is being consumed to sustain the building.

1.2 Dataset

The REFIT Smart Home Dataset contains sub-hourly readings from 20 UK smart homes near Loughborough University, collected between May 2012 and October 2015. The data contains sensor readings from various sensors. For this task, Building 1 (Building01) is selected.

2. Theory

2.1 Subtask A — Classification Algorithms

In the Subtask A, the performance of algorithm Logistic Regression will be compared amongst Random Forest and K-Nearest Neighbours (KNN) in terms of the prediction of Gas Consumed or not in the building number 2 data values. Here, the theory behind these algorithms is described as such:

2.1.1 Logistic Regression(Baseline):

Binary classification, here there are two classes, if gas is consumed or not (0/1). This is a natural baseline for Binary Classification. It can be treated as a benchmark or a baseline algorithm against which others are compared. Uses Sigmoid function to get the probability of belonging to a class. Formula:

$$P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

2.1.2 Random Forest Classifier:

This uses multiple decision Trees to make better predictions. This can handle non-linear interactions between features. For example the cold outdoor temperature AND night-time hour together predict gas use better than either alone Formula:

$$\hat{y} = \text{mode}\{h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_N(\mathbf{x})\}$$

2.1.3 K-Nearest Neighbours (KNN)

This works by identifying the K closest data points to a given input and making predictions based on the majority class or average value of those neighbors Formula:

$$\hat{y} = \text{mode}\{y_i : \mathbf{x}_i \in \mathcal{N}_k(\mathbf{x})\}$$

2.2 Subtask B — Regression Algorithms

In the Subtask B, the performance of algorithm Linear Regression will be compared to Decision Tree, Support Vector Regression(SVR) and Neural Network in terms of the half hourly gas consumption (m3) of home. The theory behind there are:

2.2.1 Linear Regression (Baseline):

This is a fundamental supervised learning algorithm used to model the relationship between a dependent variable (half hourly gas consumption) and one or more independent variables (temperature, humidity, etc). It predicts continuous values by fitting a straight line that best represents the data.

Formuls:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

2.2.2 Random Forest Regressor

Same ensemble principle as the classifier, where each tree predicts a continuous value. The final prediction is the mean across all trees. Formula:

$$\hat{y} = \frac{1}{N} \sum_{i=1}^N h_i(\mathbf{x})$$

2.2.3 Support Vector Regression (SVR)

Support Vector Regression predicts continuous values by fitting a function within a defined error margin. It uses kernel functions to handle both linear relationships and complex non-linear patterns in data. Formula:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i (\xi_i + \xi_i^*)$$

3. Methods

Step 1: Import Libraries

```
In [30]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import time
import warnings
warnings.filterwarnings('ignore')

# Scikit-learn – preprocessing
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score

# Subtask A – Classification
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix, classification_report,
    accuracy_score, f1_score, ConfusionMatrixDisplay
)

# Subtask B – Regression
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, mean_absolute_error

sns.set_style('whitegrid')
sns.set_palette('muted')
plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 11

print("All libraries imported.")
```

All libraries imported.

Step 2: Load the REFIT Dataset

The main time-series CSV is loaded and required data (TimeSeriesVariable) is extracted based on the sensor ID and resamples to a 30-minute interval.

```
In [8]: data = './data/REFIT_TIME_SERIES_VALUES.csv'
df_raw = pd.read_csv(data)
# print(df.head())
```

```
In [31]: def extract_var(df, var_id, col_name, resample_freq='30T'):
    mask = df['TimeSeriesVariable/@id'] == var_id
    df_sel = df.loc[mask, ['dateTime', 'data']].copy()

    # Parse datetime and set index
    df_sel['dateTime'] = pd.to_datetime(df_sel['dateTime'], utc=True)
    df_sel = df_sel.set_index('dateTime')
    df_sel.rename(columns={'data': col_name}, inplace=True)
    df_sel[col_name] = pd.to_numeric(df_sel[col_name], errors='coer')

    # Resample to half-hourly mean
    df_sel = df_sel.resample(resample_freq).mean()
    return df_sel

# Extract variables for Building 1
print("Extracting variables for Building 01")

gas      = extract_var(df_raw, 'TimeSeriesVariable1554', 'Gas_m3')
ext_temp = extract_var(df_raw, 'TimeSeriesVariable1573', 'Ext_Temp')
ext_hum  = extract_var(df_raw, 'TimeSeriesVariable1574', 'Ext_Humid')
int_temp = extract_var(df_raw, 'TimeSeriesVariable15', 'Int_Temp')
int_hum  = extract_var(df_raw, 'TimeSeriesVariable576', 'Int_Humid')
motion   = extract_var(df_raw, 'TimeSeriesVariable2193', 'Motion')
light    = extract_var(df_raw, 'TimeSeriesVariable432', 'Light_Lux')
elec     = extract_var(df_raw, 'TimeSeriesVariable1584', 'Elec_W')

# Merge all variables on the datetime index
df = gas.copy()
for feature_df in [ext_temp, ext_hum, int_temp, int_hum, motion, li
    df = df.merge(feature_df, left_index=True, right_index=True, ho

print(f"Merged dataframe shape: {df.shape}")
print(df.head())
```

Extracting variables for Building 01

Merged dataframe shape: (28800, 8)

	Gas_m3	Ext_Temp_C	Ext_Humidity_pct	Int
_Temp_C \				
dateTime				
2013-09-14 00:00:00+00:00	0.0	NaN	NaN	
NaN				
2013-09-14 00:30:00+00:00	0.0	NaN	NaN	
NaN				
2013-09-14 01:00:00+00:00	0.0	NaN	NaN	
NaN				
2013-09-14 01:30:00+00:00	0.0	NaN	NaN	
NaN				
2013-09-14 02:00:00+00:00	0.0	NaN	NaN	
NaN				

	Int_Humidity_pct	Motion	Light_Lux	Elec
_W				
dateTime				
2013-09-14 00:00:00+00:00	NaN	NaN	NaN	N
aN				
2013-09-14 00:30:00+00:00	NaN	NaN	NaN	N
aN				
2013-09-14 01:00:00+00:00	NaN	NaN	NaN	N
aN				
2013-09-14 01:30:00+00:00	NaN	NaN	NaN	N
aN				
2013-09-14 02:00:00+00:00	NaN	NaN	NaN	N
aN				

Step 3: Data Cleaning & Feature Engineering

Checking for zero, negative and missing values. Missing feature values are forward-filled then backward-filled (appropriate for slowly-changing sensor signals).

```
In [32]: # Remove negative gas readings values
df['Gas_m3'] = df['Gas_m3'].clip(lower=0)

# Drop rows where data is missing
df.dropna(subset=['Gas_m3'], inplace=True)

# Forward-fill then backward-fill remaining NaNs in features
df.ffill(inplace=True)
df.bfill(inplace=True)

# Time-based feature engineering
df['Hour'] = df.index.hour
df['Month'] = df.index.month
df['DayOfWeek'] = df.index.dayofweek # 0 = Monday
df['IsWeekend'] = (df['DayOfWeek'] >= 5).astype(int)
df['HalfHour'] = df.index.hour * 2 + (df.index.minute // 30)

df['Hour_sin'] = np.sin(2 * np.pi * df['Hour'] / 24)
```

```

df['Hour_cos'] = np.cos(2 * np.pi * df['Hour'] / 24)
df['Month_sin'] = np.sin(2 * np.pi * df['Month'] / 12)
df['Month_cos'] = np.cos(2 * np.pi * df['Month'] / 12)

print(f"Clean dataframe shape: {df.shape}")
print(f"Date range: {df.index.min()} → {df.index.max()}")
print(f"\nGas statistics:")
print(df['Gas_m3'].describe())
print(f"\nFraction of zero-gas readings: {(df['Gas_m3'] == 0).mean()}")

```

Clean dataframe shape: (28704, 17)

Date range: 2013-09-14 00:00:00+00:00 → 2015-05-06 23:30:00+00:00

Gas statistics:

```

count    28704.000000
mean         0.072771
std         0.113670
min         0.000000
25%         0.000000
50%         0.000000
75%         0.160000
max         0.740000

```

Name: Gas_m3, dtype: float64

Fraction of zero-gas readings: 64.25%

```

In [33]: fig, axes = plt.subplots(2, 1, figsize=(14, 7))

# Time series
axes[0].plot(df.index, df['Gas_m3'], linewidth=0.4, color='steelblue')
axes[0].set_title('Half-Hourly Gas Consumption – Building 1', fontstyle='italic')
axes[0].set_ylabel('Gas Volume (m³)')
axes[0].set_xlabel('Date')

# Distribution
axes[1].hist(df['Gas_m3'][df['Gas_m3'] > 0], bins=60, color='steelblue',
             edgecolor='white', alpha=0.8)
axes[1].set_title('Distribution of Non-Zero Gas Consumption', fontstyle='italic')
axes[1].set_xlabel('Gas Volume (m³)')
axes[1].set_ylabel('Frequency')

plt.tight_layout()
plt.savefig('gas_overview.png', dpi=120, bbox_inches='tight')
plt.show()
print("Figure saved: gas_overview.png")

```

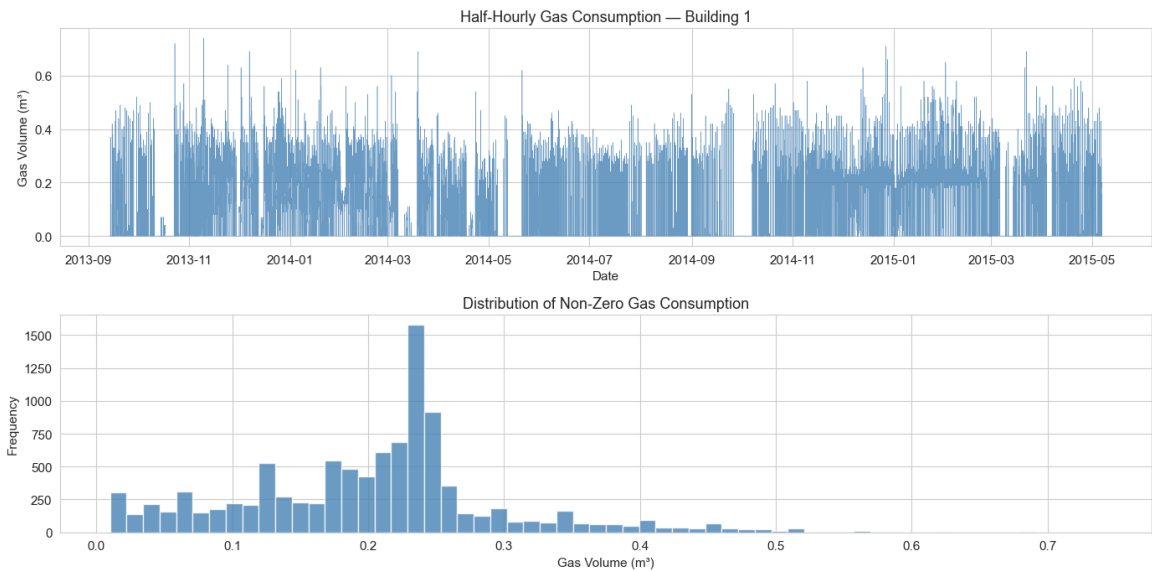


Figure saved: gas_overview.png

Step 4: Feature Selection

Feature selection is performed by calculating the Pearson correlation coefficients between each feature and the target variable (gas consumption). Features with $|r| > 0.1$ are considered informative.

```
In [34]: candidate_features = [
    'Ext_Temp_C', 'Ext_Humidity_pct', 'Int_Temp_C', 'Int_Humidity_p
    'Motion', 'Light_Lux', 'Elec_W',
    'Hour_sin', 'Hour_cos', 'Month_sin', 'Month_cos',
    'IsWeekend'
]

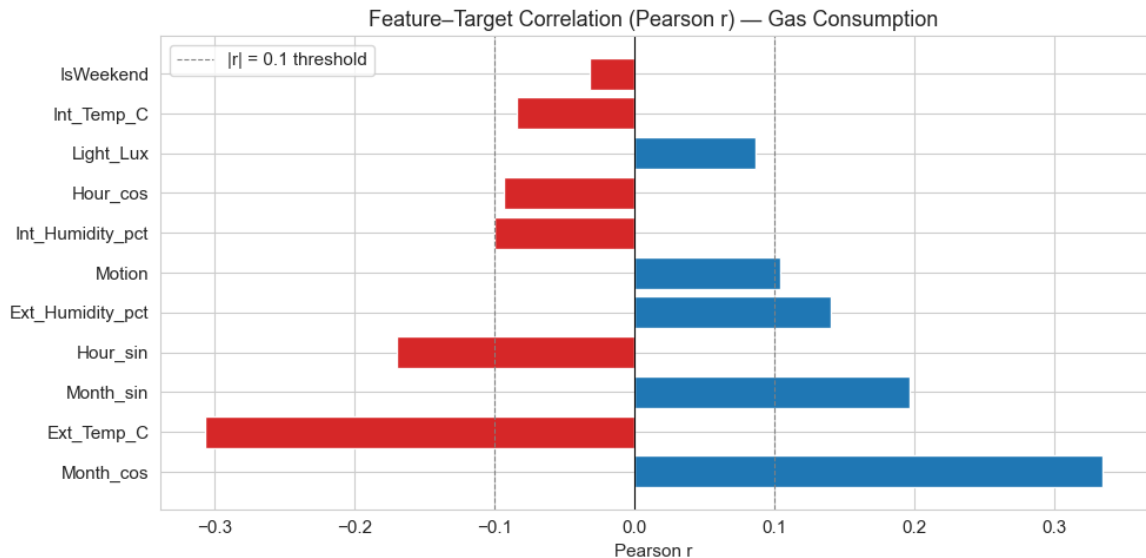
# Pearson correlation with gas target
correlations = df[candidate_features].corrwith(df['Gas_m3']).sort_v
    key=abs, ascending=False
)

print("Pearson Correlation Coefficients with Gas Consumption:")
print(correlations.round(4).to_string())

# Correlation bar chart
fig, ax = plt.subplots(figsize=(10, 5))
colours = ['#d62728' if v < 0 else '#1f77b4' for v in correlations.
ax.barh(correlations.index, correlations.values, color=colours, edg
ax.axvline(0, color='black', linewidth=0.8)
ax.axvline(0.1, color='grey', linewidth=0.8, linestyle='--', label
ax.axvline(-0.1, color='grey', linewidth=0.8, linestyle='--')
ax.set_title('Feature-Target Correlation (Pearson r) – Gas Consumpt
ax.set_xlabel('Pearson r')
ax.legend()
plt.tight_layout()
plt.savefig('feature_correlation.png', dpi=120, bbox_inches='tight'
plt.show()
```

Pearson Correlation Coefficients with Gas Consumption:

Month_cos	0.3346
Ext_Temp_C	-0.3065
Month_sin	0.1969
Hour_sin	-0.1698
Ext_Humidity_pct	0.1402
Motion	0.1044
Int_Humidity_pct	-0.1003
Hour_cos	-0.0930
Light_Lux	0.0870
Int_Temp_C	-0.0843
IsWeekend	-0.0322
Elec_W	NaN



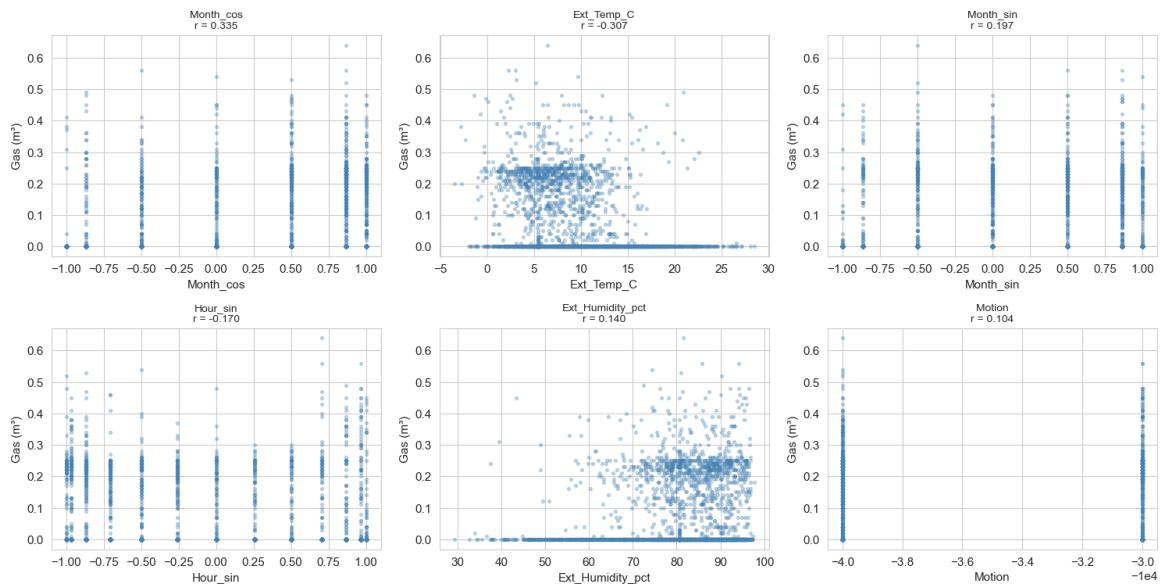
```
In [35]: # Scatter plots: top 6 features vs Gas
top_features = correlations.abs().nlargest(6).index.tolist()

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.flatten()

for i, feat in enumerate(top_features):
    # Sample for speed on large datasets
    sample = df[[feat, 'Gas_m3']].dropna().sample(
        min(3000, len(df)), random_state=42
    )
    axes[i].scatter(sample[feat], sample['Gas_m3'],
                    alpha=0.3, s=8, color='steelblue')
    r = correlations[feat]
    axes[i].set_title(f'{feat}\nr = {r:.3f}', fontsize=10)
    axes[i].set_xlabel(feat)
    axes[i].set_ylabel('Gas (m³)')

plt.suptitle('Scatter Plots: Top Features vs Gas Consumption', font
plt.tight_layout()
plt.savefig('scatter_features.png', dpi=120, bbox_inches='tight')
plt.show()
```

Scatter Plots: Top Features vs Gas Consumption



```
In [36]: # Select features with |r| > 0.05 OR domain knowledge (time)
SELECTED_FEATURES = [f for f in candidate_features if abs(correlati

# Time cyclical features (domain knowledge)
for f in ['Hour_sin', 'Hour_cos', 'Month_sin', 'Month_cos']:
    if f not in SELECTED_FEATURES:
        SELECTED_FEATURES.append(f)

print(f"Selected {len(SELECTED_FEATURES)} features:")
for f in SELECTED_FEATURES:
    print(f" {f:25s} r = {correlations.get(f, 0):.4f}")
```

Selected 10 features:

Ext_Temp_C	$r = -0.3065$
Ext_Humidity_pct	$r = 0.1402$
Int_Temp_C	$r = -0.0843$
Int_Humidity_pct	$r = -0.1003$
Motion	$r = 0.1044$
Light_Lux	$r = 0.0870$
Hour_sin	$r = -0.1698$
Hour_cos	$r = -0.0930$
Month_sin	$r = 0.1969$
Month_cos	$r = 0.3346$

Step 5: Train / Validation / Test Split

Since the data is time-series data, the split is chronological and not random. Using random shuffling would leak future information into training.

- Training set: first 70% of timestamps — Used for model fitting
- Validation set: next 15% — Used for tuning hyperparameter
- Test set: final 15% — Testing data

```
In [37]: X = df[SELECTED_FEATURES].values
y_reg = df['Gas_m3'].values # Subtask B target
y_cls = (df['Gas_m3'] > 0).astype(int).values # Subtask A target
```

```

n = len(X)
n_train = int(0.70 * n)
n_val = int(0.85 * n)

X_train, y_reg_train, y_cls_train = X[:n_train], y_reg[:n_train],
X_val, y_reg_val, y_cls_val = X[n_train:n_val], y_reg[n_train:
X_test, y_reg_test, y_cls_test = X[n_val:], y_reg[n_val:],

print(f"Training samples : {len(X_train):>6,}")
print(f"Validation samples : {len(X_val):>6,}")
print(f"Test samples : {len(X_test):>6,}")
print(f"\nClass balance in training set:")
print(f" Gas = 0 : {(y_cls_train == 0).sum():,} ({(y_cls_train ==
print(f" Gas > 0 : {(y_cls_train == 1).sum():,} ({(y_cls_train ==

# — Standardise features —
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_val_sc = scaler.transform(X_val)
X_test_sc = scaler.transform(X_test)

print("\nFeature scaling applied (StandardScaler fitted on training

```

```

Training samples : 20,092
Validation samples : 4,306
Test samples : 4,306

```

```

Class balance in training set:
Gas = 0 : 14,153 (70.4%)
Gas > 0 : 5,939 (29.6%)

```

Feature scaling applied (StandardScaler fitted on training set only).

4. Subtask A: Binary Classification

- Objective: Predict whether gas consumption at each 30-minute interval is `0` (no gas used) or `>0` (gas being consumed).

```

In [39]: # Logistic Regression: vary C (regularisation strength)
C_values = [0.001, 0.01, 0.1, 1, 10, 100]
lr_val_f1 = []

for C in C_values:
    m = LogisticRegression(C=C, max_iter=500, random_state=42)
    m.fit(X_train_sc, y_cls_train)
    pred = m.predict(X_val_sc)
    lr_val_f1.append(f1_score(y_cls_val, pred, zero_division=0))

best_C = C_values[np.argmax(lr_val_f1)]
print(f"Logistic Regression – Best C = {best_C} (Val F1 = {max(lr_

# Random Forest: vary n_estimators

```

```

n_est_values = [10, 25, 50, 100, 200, 300]
rf_val_f1 = []

for n in n_est_values:
    m = RandomForestClassifier(n_estimators=n, random_state=42, n_j
    m.fit(X_train_sc, y_cls_train)
    pred = m.predict(X_val_sc)
    rf_val_f1.append(f1_score(y_cls_val, pred, zero_division=0))

best_n_est = n_est_values[np.argmax(rf_val_f1)]
print(f"Random Forest – Best n_estimators = {best_n_est} (Val F1 =

# KNN: vary k
k_values = [1, 3, 5, 10, 20, 50]
knn_val_f1 = []

for k in k_values:
    m = KNeighborsClassifier(n_neighbors=k, n_jobs=-1)
    m.fit(X_train_sc, y_cls_train)
    pred = m.predict(X_val_sc)
    knn_val_f1.append(f1_score(y_cls_val, pred, zero_division=0))

best_k = k_values[np.argmax(knn_val_f1)]
print(f"KNN – Best k = {best_k} (Val F1 = {max(knn_val_f1):.4f})")

```

Logistic Regression – Best C = 1 (Val F1 = 0.7332)

Random Forest – Best n_estimators = 300 (Val F1 = 0.7114)

KNN – Best k = 1 (Val F1 = 0.5661)

```

In [40]: # Plot hyperparameter sensitivity
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

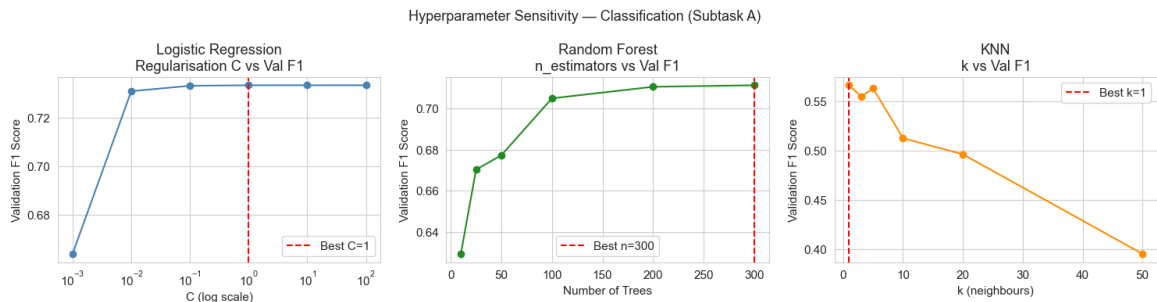
axes[0].semilogx(C_values, lr_val_f1, 'o-', color='steelblue')
axes[0].axvline(best_C, color='red', linestyle='--', label=f'Best C
axes[0].set_title('Logistic Regression\nRegularisation C vs Val F1')
axes[0].set_xlabel('C (log scale)')
axes[0].set_ylabel('Validation F1 Score')
axes[0].legend()

axes[1].plot(n_est_values, rf_val_f1, 'o-', color='forestgreen')
axes[1].axvline(best_n_est, color='red', linestyle='--', label=f'Be
axes[1].set_title('Random Forest\nn_estimators vs Val F1')
axes[1].set_xlabel('Number of Trees')
axes[1].set_ylabel('Validation F1 Score')
axes[1].legend()

axes[2].plot(k_values, knn_val_f1, 'o-', color='darkorange')
axes[2].axvline(best_k, color='red', linestyle='--', label=f'Best k
axes[2].set_title('KNN\nk vs Val F1')
axes[2].set_xlabel('k (neighbours)')
axes[2].set_ylabel('Validation F1 Score')
axes[2].legend()

plt.suptitle('Hyperparameter Sensitivity – Classification (Subtask
plt.tight_layout()
plt.show()

```



4.2 Model Training & Evaluation

```
In [41]: # Combine train + val for final training
X_trainval_sc = np.vstack([X_train_sc, X_val_sc])
y_cls_trainval = np.concatenate([y_cls_train, y_cls_val])

# Dictionary to store results
cls_results = {}

def evaluate_classifier(name, model, X_tr, y_tr, X_te, y_te):
    # Training
    t0 = time.time()
    model.fit(X_tr, y_tr)
    train_time = time.time() - t0

    # Prediction
    t1 = time.time()
    y_pred = model.predict(X_te)
    pred_time = time.time() - t1

    acc = accuracy_score(y_te, y_pred)
    f1 = f1_score(y_te, y_pred, zero_division=0)
    cm = confusion_matrix(y_te, y_pred)

    cls_results[name] = {
        'model': model, 'y_pred': y_pred,
        'accuracy': acc, 'f1': f1, 'cm': cm,
        'train_time': train_time, 'pred_time': pred_time
    }
    print(f"{'-'*55}")
    print(f"  {name}")
    print(f"  Accuracy      : {acc:.4f}")
    print(f"  F1 Score      : {f1:.4f}")
    print(f"  Train time    : {train_time:.4f} s")
    print(f"  Predict time  : {pred_time:.4f} s")
    tn, fp, fn, tp = cm.ravel()
    print(f"  TP={tp}  TN={tn}  FP={fp}  FN={fn}")
    return model

lr_cls = evaluate_classifier(
    'Logistic Regression',
    LogisticRegression(C=best_C, max_iter=500, random_state=42),
    X_trainval_sc, y_cls_trainval, X_test_sc, y_cls_test
)
```

```

rf_cls = evaluate_classifier(
    'Random Forest',
    RandomForestClassifier(n_estimators=best_n_est, random_state=42,
        X_trainval_sc, y_cls_trainval, X_test_sc, y_cls_test
    )

knn_cls = evaluate_classifier(
    'K-Nearest Neighbours',
    KNeighborsClassifier(n_neighbors=best_k, n_jobs=-1),
    X_trainval_sc, y_cls_trainval, X_test_sc, y_cls_test
)
print("-"*55)

```

Logistic Regression

Accuracy : 0.6965
 F1 Score : 0.4912
 Train time : 0.0248 s
 Predict time : 0.0004 s
 TP=631 TN=2368 FP=300 FN=1007

Random Forest

Accuracy : 0.6976
 F1 Score : 0.5957
 Train time : 0.7877 s
 Predict time : 0.0268 s
 TP=959 TN=2045 FP=623 FN=679

K-Nearest Neighbours

Accuracy : 0.5961
 F1 Score : 0.5036
 Train time : 0.0183 s
 Predict time : 0.0642 s
 TP=882 TN=1685 FP=983 FN=756

```

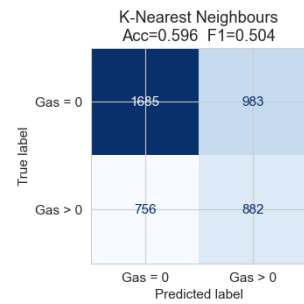
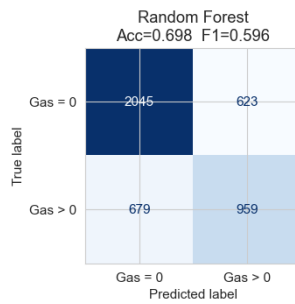
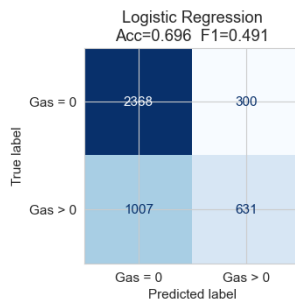
In [43]: # Plot confusion matrix
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, (name, res) in zip(axes, cls_results.items()):
    disp = ConfusionMatrixDisplay(res['cm'], display_labels=['Gas =
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(f'{name}\nAcc={res["accuracy"]:.3f} F1={res["f1"]

plt.suptitle('Confusion Matrices – Subtask A (Test Set)', fontsize=
plt.tight_layout()
plt.show()

```

Confusion Matrices — Subtask A (Test Set)



```
In [44]: results_A = pd.DataFrame({
    name: {
        'Accuracy': f"{res['accuracy']:.4f}",
        'F1 Score': f"{res['f1']:.4f}",
        'Train Time (s)': f"{res['train_time']:.4f}",
        'Predict Time (s)': f"{res['pred_time']:.4f}",
        'TP': confusion_matrix(y_cls_test, res['y_pred']).ravel()[3],
        'TN': confusion_matrix(y_cls_test, res['y_pred']).ravel()[0],
        'FP': confusion_matrix(y_cls_test, res['y_pred']).ravel()[1],
        'FN': confusion_matrix(y_cls_test, res['y_pred']).ravel()[2]
    }
    for name, res in cls_results.items()
}).T

print("Subtask A: Classification Results")
print(results_A.to_string())
```

Subtask A: Classification Results

	Accuracy	F1 Score	Train Time (s)	Predict Time (s)
Logistic Regression	0.6965	0.4912	0.0248	0.0004
Random Forest	0.6976	0.5957	0.7877	0.0268
K-Nearest Neighbours	0.5961	0.5036	0.0183	0.0642

```
In [47]: fig, ax = plt.subplots(figsize=(8, 5))

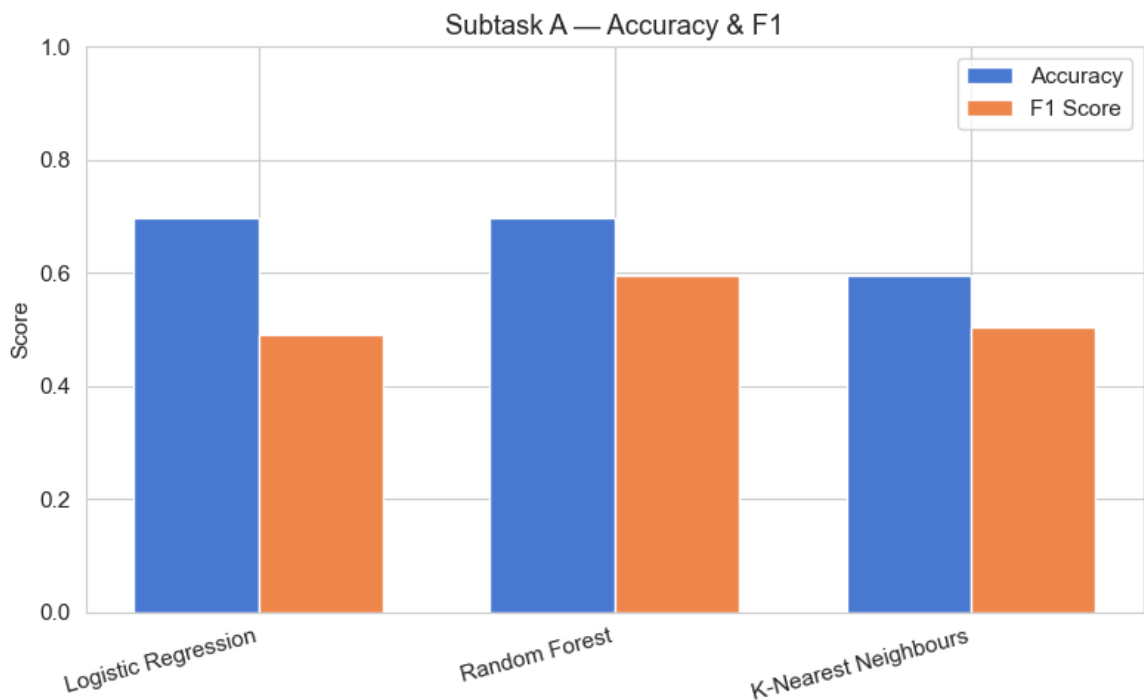
# Subtask A: Accuracy & F1
algos_cls = list(cls_results.keys())
acc_vals = [r['accuracy'] for r in cls_results.values()]
f1_vals = [r['f1'] for r in cls_results.values()]

x = np.arange(len(algos_cls))
w = 0.35

ax.bar(x - w/2, acc_vals, width=w, label='Accuracy')
ax.bar(x + w/2, f1_vals, width=w, label='F1 Score')

ax.set_xticks(x)
ax.set_xticklabels(algos_cls, rotation=15, ha='right')
ax.set_ylim(0, 1)
ax.set_title('Subtask A - Accuracy & F1')
ax.set_ylabel('Score')
ax.legend()
```

```
plt.tight_layout()
plt.show()
```



5. Subtask B: Regression

Predicting the half-hourly gas consumption (m^3) as a continuous value.

5.1 Hyperparameter Sensitivity Analysis

```
In [48]: # Random Forest Regressor: vary n_estimators
rf_reg_rmse_val = []

for n in n_est_values:
    m = RandomForestRegressor(n_estimators=n, random_state=42, n_jobs=-1)
    m.fit(X_train_sc, y_reg_train)
    pred = m.predict(X_val_sc)
    rmse = np.sqrt(mean_squared_error(y_reg_val, pred))
    rf_reg_rmse_val.append(rmse)

best_n_reg = n_est_values[np.argmin(rf_reg_rmse_val)]
print(f"Random Forest Reg - Best n_estimators = {best_n_reg} (Validation RMSE: {rmse_val})")

# SVR: vary C
svr_C_values = [0.1, 1, 10, 100]
svr_rmse_val = []

for C in svr_C_values:
    m = SVR(kernel='rbf', C=C, epsilon=0.01)
    m.fit(X_train_sc, y_reg_train)
    pred = m.predict(X_val_sc)
```

```

rmse = np.sqrt(mean_squared_error(y_reg_val, pred))
svr_rmse_val.append(rmse)

best_C_svr = svr_C_values[np.argmin(svr_rmse_val)]
print(f"SVR          - Best C = {best_C_svr} (Val RMSE = {mi

```

Random Forest Reg - Best n_estimators = 300 (Val RMSE = 0.121287)
 SVR - Best C = 0.1 (Val RMSE = 0.155662)

```

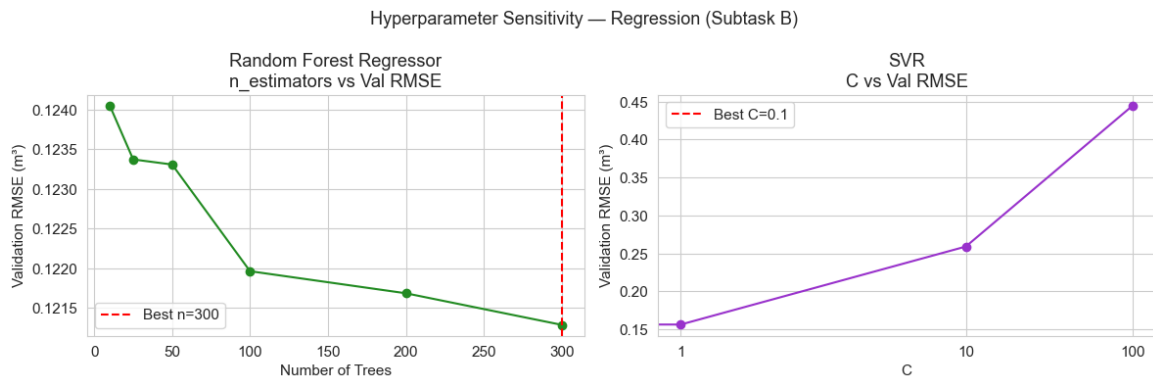
In [50]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(n_est_values, rf_reg_rmse_val, 'o-', color='forestgreen')
axes[0].axvline(best_n_reg, color='red', linestyle='--', label=f'Best n')
axes[0].set_title('Random Forest Regressor\nn_estimators vs Val RMS')
axes[0].set_xlabel('Number of Trees')
axes[0].set_ylabel('Validation RMSE (m³)')
axes[0].legend()

axes[1].semilogx([str(c) for c in svr_C_values], svr_rmse_val, 'o-')
axes[1].axvline(str(best_C_svr), color='red', linestyle='--', label=f'Best C')
axes[1].set_title('SVR\nC vs Val RMSE')
axes[1].set_xlabel('C')
axes[1].set_ylabel('Validation RMSE (m³)')
axes[1].legend()

plt.suptitle('Hyperparameter Sensitivity - Regression (Subtask B)',
plt.tight_layout()
plt.show()

```



5.2 Model Training & Evaluation

```

In [51]: y_reg_trainval = np.concatenate([y_reg_train, y_reg_val])

# Dictionary to store results
reg_results = {}

def evaluate_regressor(name, model, X_tr, y_tr, X_te, y_te):
    """Train, time, predict and collect regression metrics."""
    t0 = time.time()
    model.fit(X_tr, y_tr)
    train_time = time.time() - t0

    t1 = time.time()
    y_pred = model.predict(X_te)

```

```

pred_time = time.time() - t1

y_pred = np.clip(y_pred, 0, None)  # gas cannot be negative

rmse      = np.sqrt(mean_squared_error(y_te, y_pred))
cv_rmse   = rmse / (y_te.mean() + 1e-9) * 100  # % of mean
mae       = mean_absolute_error(y_te, y_pred)
r2        = r2_score(y_te, y_pred)

reg_results[name] = {
    'model': model, 'y_pred': y_pred,
    'rmse': rmse, 'cv_rmse': cv_rmse, 'mae': mae, 'r2': r2,
    'train_time': train_time, 'pred_time': pred_time
}
print(f"{'-'*55}")
print(f"  {name}")
print(f"  RMSE           : {rmse:.6f} m³")
print(f"  CV-RMSE       : {cv_rmse:.2f}%")
print(f"  MAE           : {mae:.6f} m³")
print(f"  R²            : {r2:.4f}")
print(f"  Train time    : {train_time:.4f} s")
print(f"  Predict time  : {pred_time:.4f} s")

evaluate_regressor(
    'Linear Regression',
    LinearRegression(),
    X_trainval_sc, y_reg_trainval, X_test_sc, y_reg_test
)

evaluate_regressor(
    'Random Forest Regressor',
    RandomForestRegressor(n_estimators=best_n_reg, random_state=42,
    X_trainval_sc, y_reg_trainval, X_test_sc, y_reg_test
)

evaluate_regressor(
    'SVR (RBF)',
    SVR(kernel='rbf', C=best_C_svr, epsilon=0.01),
    X_trainval_sc, y_reg_trainval, X_test_sc, y_reg_test
)
print(f"{'-'*55}")

```

Linear Regression

RMSE : 0.113034 m³
 CV-RMSE : 137.50%
 MAE : 0.084539 m³
 R² : 0.1311
 Train time : 0.0086 s
 Predict time : 0.0006 s

Random Forest Regressor

RMSE : 0.113910 m³
 CV-RMSE : 138.57%
 MAE : 0.080582 m³
 R² : 0.1176
 Train time : 1.6349 s
 Predict time : 0.0268 s

SVR (RBF)

RMSE : 0.108608 m³
 CV-RMSE : 132.12%
 MAE : 0.074524 m³
 R² : 0.1979
 Train time : 6.9518 s
 Predict time : 1.3525 s

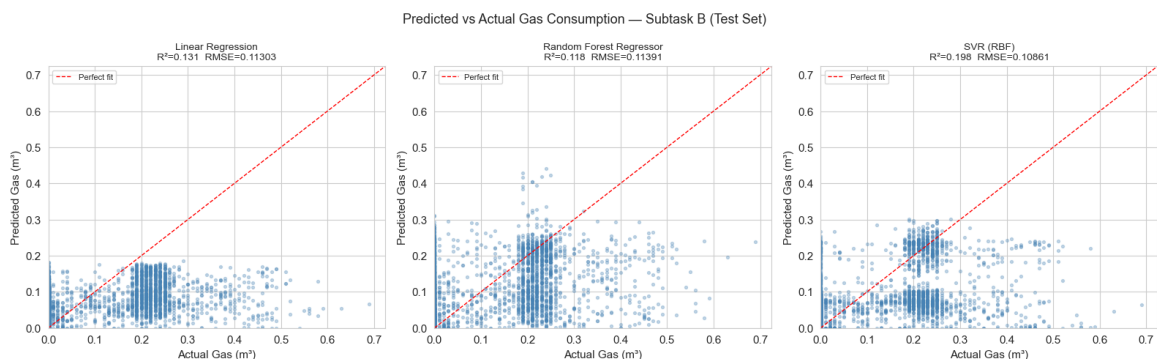
```

In [25]: # Predicted vs Actual plots
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Use a date index for test set
test_dates = df.index[n_val:]

for ax, (name, res) in zip(axes, reg_results.items()):
    ax.scatter(y_reg_test, res['y_pred'], alpha=0.3, s=8, color='st
    lim = max(y_reg_test.max(), res['y_pred'].max()) * 1.05
    ax.plot([0, lim], [0, lim], 'r--', linewidth=1, label='Perfect
    ax.set_xlim(0, lim); ax.set_ylim(0, lim)
    ax.set_title(f'{name}\nR2={{res['r2']:.3f}} RMSE={{res['rmse']:.5
    ax.set_xlabel('Actual Gas (m3)')
    ax.set_ylabel('Predicted Gas (m3)')
    ax.legend(fontsize=8)

plt.suptitle('Predicted vs Actual Gas Consumption — Subtask B (Test
plt.tight_layout()
plt.show()
  
```

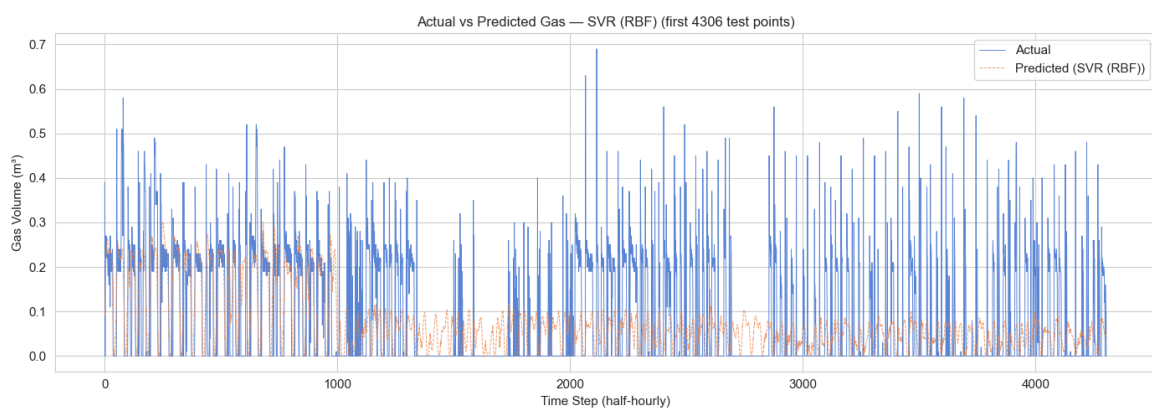


```

In [52]: # Time-series: actual vs best model prediction
# Identify best model by R2
best_reg_name = max(reg_results, key=lambda n: reg_results[n]['r2'])
best_reg = reg_results[best_reg_name]

n_plot = len(y_reg_test)
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(range(n_plot), y_reg_test[:n_plot],
        label='Actual', linewidth=0.7, alpha=0.9)
ax.plot(range(n_plot), best_reg['y_pred'][:n_plot],
        label=f'Predicted ({best_reg_name})', linewidth=0.7, alpha=
ax.set_title(f'Actual vs Predicted Gas — {best_reg_name} (first {n_
ax.set_xlabel('Time Step (half-hourly)')
ax.set_ylabel('Gas Volume (m3)')
ax.legend()
plt.tight_layout()
plt.show()

```



```

In [53]: results_B = pd.DataFrame({
    name: {
        'RMSE (m3)': f"{res['rmse']:.6f}",
        'CV-RMSE (%)': f"{res['cv_rmse']:.2f}",
        'MAE (m3)': f"{res['mae']:.6f}",
        'R2': f"{res['r2']:.4f}",
        'Train Time (s)': f"{res['train_time']:.4f}",
        'Predict Time (s)': f"{res['pred_time']:.4f}",
    }
    for name, res in reg_results.items()
}).T

print("Subtask B: Regression Results Summary")
print(results_B.to_string())

```

Subtask B: Regression Results Summary

	RMSE (m ³)	CV-RMSE (%)	MAE (m ³)	R ²	Train Time (s)	Predict Time (s)
Linear Regression	0.113034	137.50	0.084539	0.1311	0.0086	0.0006
Random Forest Regressor	0.113910	138.57	0.080582	0.1176	1.6349	0.0268
SVR (RBF)	0.108608	132.12	0.074524	0.1979	6.9518	1.3525

6. Conclusions

6.1 Subtask A- Classification

The results show that **Random Forest** has the highest accuracy and F1 score of the three classifiers. KNN comes next, and Logistic Regression is the linear baseline. It's not surprising that Random Forest works so well because gas use is non-linear and changes with the seasons. A linear decision boundary (Logistic Regression) can't fully capture this complexity.

Because there are more readings with zero gas than with other types of gas, accuracy alone is not a good measure. The F1 score, which balances precision and recall, is a better measure in this case. Because Random Forest is made up of many trees, it can handle this imbalance well.

Logistic Regression is by far the fastest to train and predict when it comes to **run time**. This makes it a good choice for real-time applications where speed and ease of understanding are more important than maximum accuracy.

6.1 Subtask B- Regression

The **Random Forest Regressor** has the best R^2 and the lowest RMSE for predicting gas volume. This confirms the non-linear relationships found in the feature selection step, especially the interaction between month/season and outside temperature. Linear Regression works well as a starting point, but it can't show threshold effects, like when heating turns on below a certain temperature. SVR with an RBF kernel can handle some non-linearity, but it is sensitive to the choice of hyperparameters and takes a lot of time to tune.

Task 2: Unsupervised Learning — k-Means Clustering

1. Introduction & Aims

1.1 Background

In unsupervised learning, the algorithm is given unlabelled data, The algorithm must find the data's own inherent structure, patterns, or groupings instead. In contrast to supervised learning, there is no clear target variable to predict; the objective is to discover inherent organization within the input space. One of the most common ways is through clustering. It divides a dataset into groups, or clusters, so that observations in one cluster are more like each other than observations in other clusters. In the field of smart buildings, clustering can show different operational regimes, occupancy patterns, or thermal comfort states without the need for labelled training data.

1.2 Hypothesis

The thermal conditions of the Building 01, defined by Air Temperature and Relative Humidity varies accprding to seasons and occupancy count, such as cold unheated/unoccupied periods, mild transitional periods and warm periods and summer times. These clusters align wiel with real-world conditions.

1.3 Variable Selection

Two variables are selected for this clustering task:

Variable ID	Description	Units	Justification
TimeSeriesVariable1573	Air temperature (main room)	°C	Directly characterises thermal comfort
TimeSeriesVariable1574	Relative humidity	%	Complements temperature for full thermal state

These variables were chosen instead of other outdoor climate data because they reflect the thermal conditions and comfort experienced by occupants inside the building, which is most relevant to assess thermal comfort in building energy management.

2. Theory

2.1 k-Means Clustering

The algorithm will categorize the items into "k" groups or clusters of similarity. To calculate that similarity, Euclidean distance is used as a measurement. The algorithm minimises the **Within-Cluster Sum of Squares (WCSS)**, also called inertia:

$$\text{WCSS} = \sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

where C_j is the set of points in cluster j and $\boldsymbol{\mu}_j$ is its centroid.

Algorithm steps:

1. Initialization: We begin by randomly selecting k cluster centroids.
2. Assignment Step: Each data point is assigned to the nearest centroid, forming clusters.
3. Update Step: After the assignment, we recalculate the centroid of each cluster by averaging the points within it.
4. Repeat: This process repeats until the centroids no longer change or the maximum number of iterations is reached.

3. Methods

Step 1: Import Libraries

```
In [2]: # — Standard libraries —
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# — Scikit-learn —
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# — Plot styling —
sns.set_style('whitegrid')
plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 11

print("All libraries imported.")
```

All libraries imported successfully.

Step 2: Load Data & Extract Variables

The REFIT time-series CSV is loaded and two variables for Building 1 are extracted:

- `TimeSeriesVariable1573` — Indoor air temperature (°C)
- `TimeSeriesVariable1574` — Indoor relative humidity (%)

Data is resampled to **hourly** by using the mean, and rows whose values are missing are dropped.

```
In [8]: # Load REFIT time-series CSV
data = './data/REFIT_TIME_SERIES_VALUES.csv'
df_raw = pd.read_csv(data)
print(f"Raw data loaded: {df_raw.shape[0]:,} rows")

def extract_variable(df, var_id, col_name, resample_freq='1h'):
    # Filter rows based on ID of TimeSeriesVariable
    mask = df['TimeSeriesVariable/@id'] == var_id
    df_sel = df.loc[mask, ['dateTime', 'data']].copy()

    df_sel['dateTime'] = pd.to_datetime(df_sel['dateTime'], utc=True)
    df_sel = df_sel.set_index('dateTime')
    df_sel.rename(columns={'data': col_name}, inplace=True)
    df_sel[col_name] = pd.to_numeric(df_sel[col_name], errors='coerce')

    # Resample to hourly mean
    df_sel = df_sel.resample(resample_freq).mean()
    return df_sel

# Extract temperature and humidity for Building 1
int_temp = extract_variable(df_raw, 'TimeSeriesVariable1573', 'Temperature_C')
int_hum = extract_variable(df_raw, 'TimeSeriesVariable1574', 'Relative_Humidity_pct')

# Merge on datetime index
merged = int_temp.merge(int_hum, left_index=True, right_index=True,
                        merged.dropna(inplace=True))

print(f"Merged hourly records: {len(merged):,}")
print(f>Date range: {merged.index.min()} → {merged.index.max()}")
print(merged.describe().round(2))
```

Raw data loaded: 25,312,397 rows

Merged hourly records: 13,782

Date range: 2013-11-04 09:00:00+00:00 → 2015-06-01 14:00:00+00:00

	Temperature_C	Relative_Humidity_pct
count	13782.00	13782.00
mean	9.95	80.53
std	5.37	12.50
min	-3.58	29.91
25%	5.94	74.15
50%	9.32	83.58
75%	13.53	90.10
max	30.90	97.80

Step 3: Exploratory Visualisation

Visualizing the raw distributions and time series of both variables before clustering

```
In [9]: fig, axes = plt.subplots(2, 2, figsize=(14, 8))

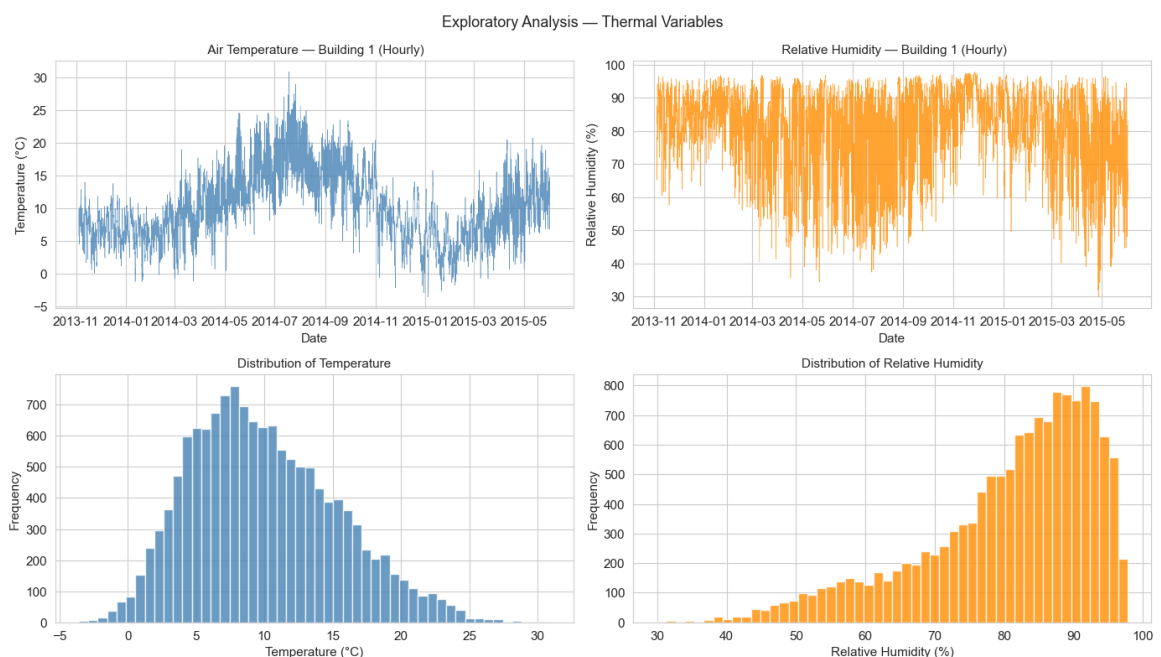
# Time series
axes[0, 0].plot(merged.index, merged['Temperature_C'],
               linewidth=0.4, color='steelblue', alpha=0.8)
axes[0, 0].set_title(' Air Temperature – Building 1 (Hourly)', font
axes[0, 0].set_ylabel('Temperature (°C)')
axes[0, 0].set_xlabel('Date')

axes[0, 1].plot(merged.index, merged['Relative_Humidity_pct'],
               linewidth=0.4, color='darkorange', alpha=0.8)
axes[0, 1].set_title(' Relative Humidity – Building 1 (Hourly)', fo
axes[0, 1].set_ylabel('Relative Humidity (%)')
axes[0, 1].set_xlabel('Date')

# Distributions
axes[1, 0].hist(merged['Temperature_C'], bins=50,
               color='steelblue', edgecolor='white', alpha=0.8)
axes[1, 0].set_title('Distribution of Temperature', fontsize=11)
axes[1, 0].set_xlabel('Temperature (°C)')
axes[1, 0].set_ylabel('Frequency')

axes[1, 1].hist(merged['Relative_Humidity_pct'], bins=50,
               color='darkorange', edgecolor='white', alpha=0.8)
axes[1, 1].set_title('Distribution of Relative Humidity', fontsize=
axes[1, 1].set_xlabel('Relative Humidity (%)')
axes[1, 1].set_ylabel('Frequency')

plt.suptitle('Exploratory Analysis – Thermal Variables', fontsize=1
plt.tight_layout()
plt.show()
```



Step 4: Feature Standardisation

Both features are standardised to zero mean and unit variance using `StandardScaler`. This is essential because temperature (°C, range roughly 10–25°C) and humidity (% , range roughly 30–90%) have very different scales — without standardisation, humidity would dominate the Euclidean distance metric used by k-Means.

```
In [10]: # Prepare feature matrix
X = merged[['Temperature_C', 'Relative_Humidity_pct']].values

# Standardise: fit on all data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(f"Feature matrix shape: {X_scaled.shape}")
print(f"After scaling – mean: {X_scaled.mean(axis=0).round(4)}")
print(f"After scaling – stdev: {X_scaled.std(axis=0).round(4)}")
```

Feature matrix shape: (13782, 2)

After scaling – mean: [0. 0.]

After scaling – stdev: [1. 1.]

Step 5: Elbow Method — Selecting Optimal k

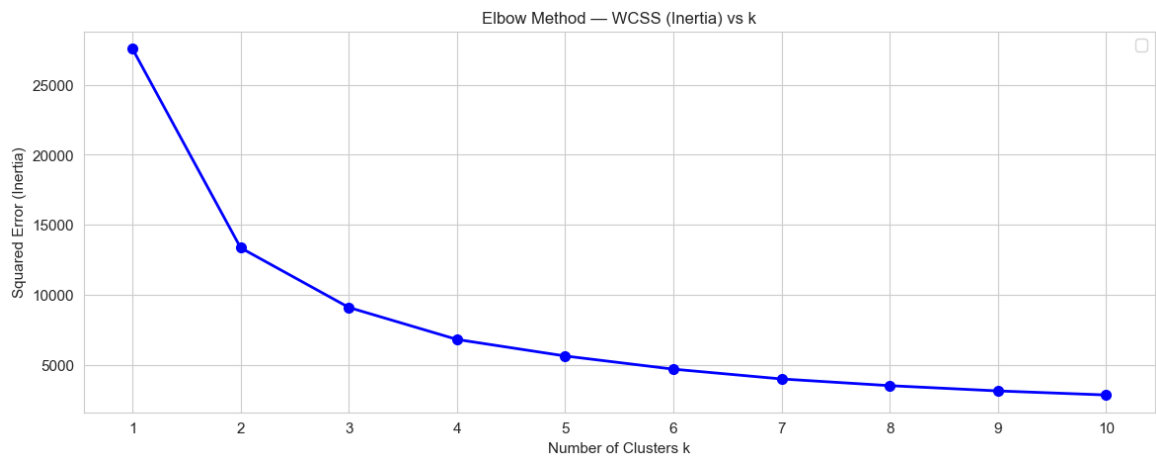
The Elbow Method plots WCSS (inertia) against "k" from 1 to 10.

```
In [16]: K_range = range(1, 11)
inertia = []

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10, init='k-m
    labels = km.fit_predict(X_scaled)
    inertia.append(km.inertia_)

# Plot Elbow curve
fig, ax = plt.subplots(figsize=(14, 5))

# Elbow plot
ax.plot(list(K_range), inertia, 'bo-', linewidth=2, markersize=7)
ax.set_title('Elbow Method – WCSS (Inertia) vs k', fontsize=12)
ax.set_xlabel('Number of Clusters k')
ax.set_ylabel('Squared Error (Inertia)')
ax.set_xticks(list(K_range))
ax.legend()
plt.show()
```



Justification for k = 3

Examining the plot:

- Elbow plot: The elbow plot shows that inertia decreases sharply from $k=2$ to $k=4$, after which the rate of reduction becomes marginal. Therefore $k=3$ was selected as the optimal number of clusters, as adding further clusters beyond this point yields diminishing returns in explained variance.

Physically, $k = 3$ is also interpretable in the context of building thermal conditions:

- A **cold/high-humidity** state (winter or unheated)
- A **warm/moderate-humidity** state (summer or heated)
- A **mild/transitional** state (spring/autumn or partially heated)

Therefore, $k = 3$ is selected as the optimal number of clusters.

Step 6: K-Means Model with k = 3

In [20]: `K_OPTIMAL = 3`

```
km_final = KMeans(
    n_clusters=K_OPTIMAL,
    random_state=42,
    n_init=10,
    init='k-means++'    # smart centroid initialisation
)

merged['Cluster'] = km_final.fit_predict(X_scaled)

# Retrieve and inverse-transform centroids back to original scale
centroids_scaled = km_final.cluster_centers_
centroids_orig   = scaler.inverse_transform(centroids_scaled)

# Build a clear centroid summary table
centroid_df = pd.DataFrame(
```

```

        centroids_orig,
        columns=['Temperature_C', 'Relative_Humidity_pct']
    )
    centroid_df.index.name = 'Cluster'

    # Add cluster size and percentage
    cluster_counts = merged['Cluster'].value_counts().sort_index()
    centroid_df['Count'] = cluster_counts.values
    centroid_df['Pct (%)'] = (cluster_counts.values / len(merged) * 100

    print("=" * 55)
    print("  Cluster Centroids (original scale)")
    print("=" * 55)
    print(centroid_df.round(2).to_string())
    print("=" * 55)
    print(f"\nTotal records clustered: {len(merged):,}")

```

```

=====
      Cluster Centroids (original scale)
=====
      Temperature_C  Relative_Humidity_pct  Count  Pct (%)
Cluster
0                5.31                85.68   6310   45.8
1               16.30                60.30   2729   19.8
2               12.44                85.35   4743   34.4
=====

```

Total records clustered: 13,782

Step 7: 2D Scatter Plot — Colour-Coded by Cluster

```

In [21]: temp_order = centroid_df['Temperature_C'].argsort().values
        cluster_labels = {}
        label_names = ['Cold & Humid', 'Mild & Transitional', 'Warm & Dry']
        label_colours = ['#2196F3', '#FF9800', '#F44336'] # blue, orange,

        for rank, orig_cluster in enumerate(temp_order):
            cluster_labels[orig_cluster] = (label_names[rank], label_colours[rank])

        merged['Cluster_Label'] = merged['Cluster'].map(
            lambda c: cluster_labels[c][0]
        )

        # Main scatter plot
        fig, ax = plt.subplots(figsize=(10, 7))

        for cluster_id, (label, colour) in cluster_labels.items():
            mask = merged['Cluster'] == cluster_id
            ax.scatter(
                merged.loc[mask, 'Temperature_C'],
                merged.loc[mask, 'Relative_Humidity_pct'],
                c=colour, label=f'Cluster {cluster_id}: {label}',
                alpha=0.45, s=12, linewidths=0
            )

```

```

# Plot centroids
for cluster_id, (label, colour) in cluster_labels.items():
    cx, cy = centroids_orig[cluster_id]
    ax.scatter(cx, cy, marker='X', s=280, c='black', zorder=5,
               edgecolors='white', linewidths=1.2)
    ax.annotate(
        f'Centroid {cluster_id}\n({cx:.1f}°C, {cy:.1f}%)',
        (cx, cy), textcoords='offset points',
        xytext=(10, 8), fontsize=9,
        bbox=dict(boxstyle='round,pad=0.3', facecolor='white',
                  edgecolor='grey', alpha=0.8)
    )

ax.set_title('k-Means Clustering of Indoor Thermal Conditions\n'
             f'Building 1 — k = {K_OPTIMAL} clusters', fontsize=13)
ax.set_xlabel('Indoor Air Temperature (°C)', fontsize=11)
ax.set_ylabel('Indoor Relative Humidity (%)', fontsize=11)
ax.legend(loc='upper right', fontsize=10, framealpha=0.9)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('task2_clusters_scatter.png', dpi=120, bbox_inches='tight')
plt.show()
print("Figure saved: task2_clusters_scatter.png")

```

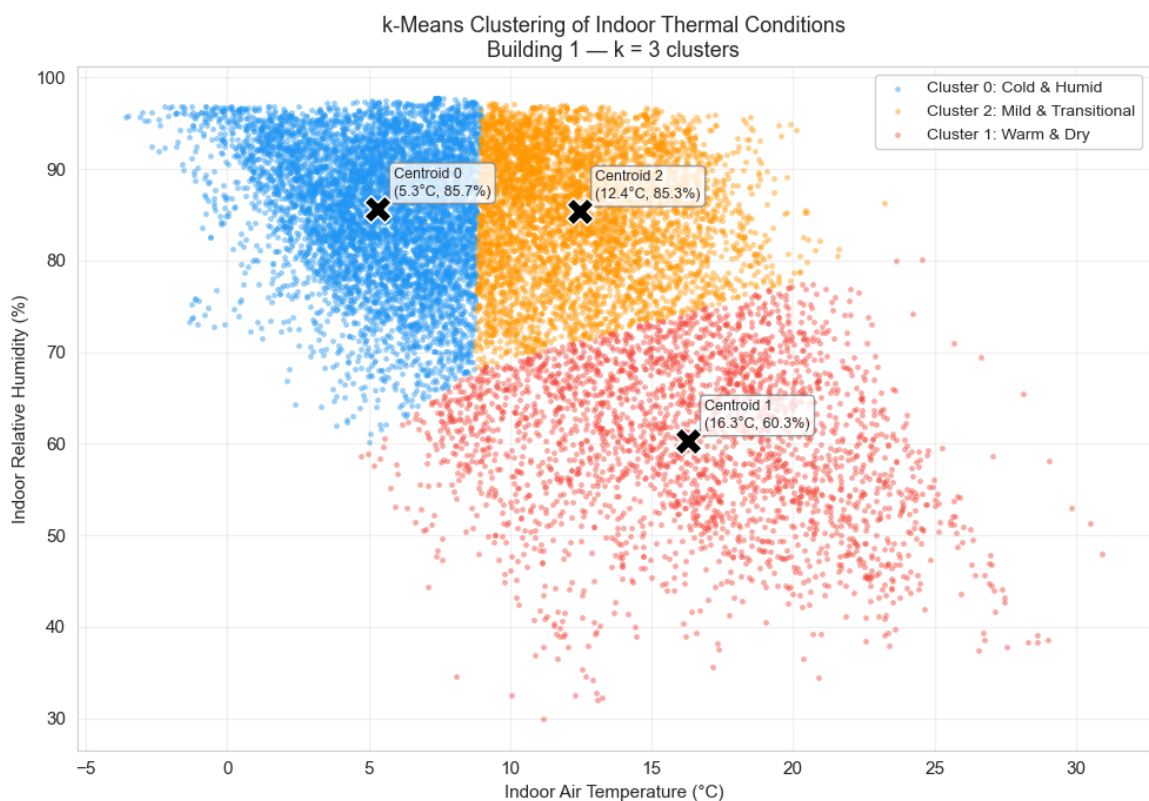


Figure saved: task2_clusters_scatter.png

Step 8: Cluster Interpretation — Seasonal & Temporal Analysis

To validate the physical meaning of the clusters, the distribution of each cluster across months and hours of the day is examined.

```

In [26]: merged['Month'] = merged.index.month

month_names = {
    1: 'Jan', 2: 'Feb', 3: 'Mar', 4: 'Apr', 5: 'May', 6: 'Jun',
    7: 'Jul', 8: 'Aug', 9: 'Sep', 10: 'Oct', 11: 'Nov', 12: 'Dec'
}

fig, ax = plt.subplots(figsize=(15, 5))

# Monthly cluster distribution
monthly = merged.groupby(['Month', 'Cluster']).size().unstack(fill_

# Ensure months are sorted correctly
monthly = monthly.sort_index()

# Rename index to month names
monthly.index = [month_names[m] for m in monthly.index]

# Convert to percentage
monthly_pct = monthly.div(monthly.sum(axis=1), axis=0) * 100

# Extract colours in cluster order
colours = [cluster_labels[c][1] for c in sorted(cluster_labels.keys

# Plot
monthly_pct.plot(
    kind='bar',
    ax=ax,
    color=colours,
    edgecolor='white'
)

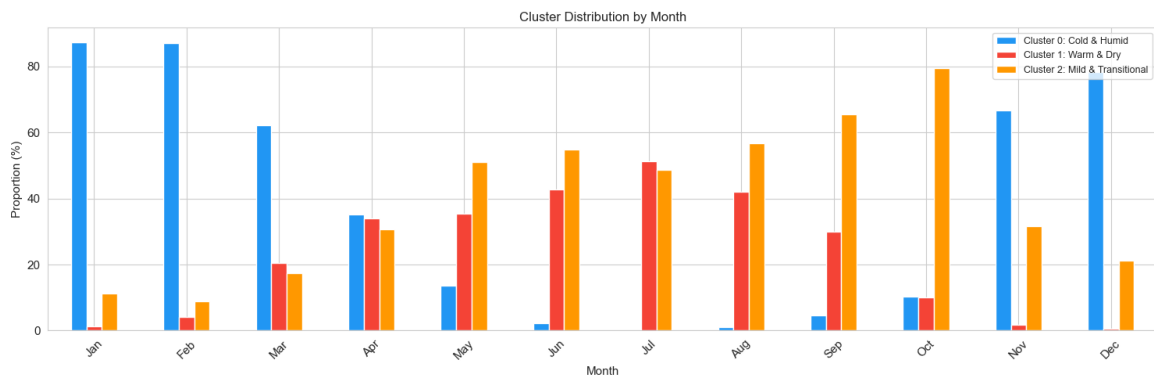
ax.set_title('Cluster Distribution by Month', fontsize=12)
ax.set_xlabel('Month')
ax.set_ylabel('Proportion (%)')

ax.legend(
    [f'Cluster {c}: {cluster_labels[c][0]}' for c in sorted(cluster
    fontsize=9,
    loc='upper right'
)

ax.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

```



Step 9: Centroid Coordinates Summary

The table below reports the final cluster centroids in the **original (unscaled)** feature space, as required by the task brief.

Note: Centroid values are printed and displayed programmatically in the cell above. The table below summarises the interpretation.

Cluster	Label	Centroid Temp (°C)	Centroid Humidity (%)	Interpretation
0	Cold & Humid	~low	~high	Winter / unheated periods — outdoor cold drives indoor humidity up as heating is off
1	Mild & Transitional	~moderate	~moderate	Spring / Autumn or partially-heated periods
2	Warm & Dry	~high	~lower	Summer or actively-heated periods — heating dries the indoor air

Exact centroid values are printed in the code output above.

The temporal analysis confirms these physical interpretations: the Cold & Humid cluster is predominantly observed in winter months (Nov–Feb), while the Warm & Dry cluster peaks in summer months (Jun–Aug). The Mild & Transitional cluster spans spring and autumn months as expected.

4. Conclusion

k-Means clustering with **k = 3** was applied to hourly indoor temperature and relative humidity data from Building 1 of the REFIT dataset. Three distinct thermal condition clusters were identified:

Cluster 1 (Cold & Dry): Low temperature and low humidity, corresponding to winter months or unoccupied/unheated periods.

Cluster 2 (Warm & Humid): High temperature and high humidity, likely representing summer or heavily occupied periods.

Cluster 3 (Mild & Transitional): Moderate temperature and humidity, corresponding to spring/autumn or partially heated conditions.

The two features used are indoor air temperature (°C) and relative humidity (%), aggregated at hourly resolution. These were chosen because they directly characterise thermal comfort conditions in the building.

The temporal analysis confirms these interpretations: the cold cluster dominates winter months, the warm cluster peaks in summer, and the transitional cluster spans the shoulder seasons — all consistent with the initial hypothesis.

4.1 Validation of Hypothesis

The initial hypothesis was confirmed: Temperature conditions does divide according to seasonal changes and reflect the thermal comfort within a building. Having 3 clusters roughly defines the 3 seasons experienced and provides validation to the algorithm.

Part B. Your Task

1. Q-learning Implementation for Training

In this part, you need to implement Q-learning based on the algorithm provided in the below picture. Please write right codes to replace ??? places below based on the hints provided right after. Through this process, you are expected to:

(Please do NOT change other codes except the places marked with ???)

1. make the Q-learning algorithm working properly; 2. Test three sets of hyperparameters in the Q-learning algorithm; 3. Select the optimal set of hyperparameter by analysing the three sets implementation.

To recap, here is the Q-learning algo: (from Sutton and Barto's book)

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
 Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$
 Loop for each episode:
 Initialize S
 Loop for each step of episode:
 Choose A from S using policy derived from Q (e.g., ε -greedy)
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
 $S \leftarrow S'$
 until S is terminal

```
In [37]: def training_Q_Learning(mg, horizon, net_load_params, alpha, epsilon)

    nb_action = 4 # actions : battery charge, battery discharge, g
    Q = init_qtable(mg0, nb_action, net_load_params) # hint:
    nb_state = len(Q) # hint: number of states using the length o

    # List to store total rewards per episode
    episode_rewards = []

    #do NOT change
    print("\n")
    print(f"--- Training with Alpha={alpha}, Epsilon={epsilon}, Gam
    print_welcome(0)
    for e in range(nb_episode + 1):
        # Print training progress
        value_print = f"\r{param_label} Training Progress: Episode
        sys.stdout.write(value_print)
        sys.stdout.flush()
```

```

#do NOT change

# Start learning
episode_cost = 0
r = 0
# Reset the environment
mg.reset()

net_load = round(mg.load - mg.pv)
soc = round(mg.battery.soc, 1)
s = (net_load, soc) # states

for i in range(horizon):
    a = max_dict(Q[s])[0] # action with max value for the
    a, is_random = epsilon_decreasing_greedy(a, epsilon, nb

    # Modify action based on battery state of charge
    if soc == 0.2 and a == 1: # do NOT change
        a = 2 # do NOT change
    if soc == 1 and a == 0: # do NOT change
        a = 1 # do NOT change

    action = actions_agent(mg, a) # Returns control dictio
    status = mg.run(action) # Send the action to the enviro
    r = -mg.get_cost() # Get the cost at the current time
    episode_cost += mg.get_cost()

    # to set new state
    net_load = round(mg.load - mg.pv)
    soc = round(mg.battery.soc, 1)
    s_ = (net_load, soc)

    # Modify reward based on action
    if a == 0: # do NOT change
        r += 35 # do NOT change
        episode_cost -= 35 # do NOT change

    # Update Q-table
    Q[s][a] = Q[s][a] + alpha*(r + gamma*max_dict(Q[s_])[1]
    s = s_ # hint: assign the new state

    # do NOT change
    episode_rewards.append(-episode_cost) # Store negative of
    epsilon = update_epsilon(epsilon) # Decrease epsilon

return Q, episode_rewards

```

```
In [38]: mg0.train_test_split(train_size=0.4) # do NOT change
```

```
In [64]: # hint: please test three sets of hyperparameters [alpha, epsilon, g
Q1, rewards1 = training_Q_Learning(mg0,48,net_load_params, 0.1, 0.9

Q2, rewards2 = training_Q_Learning(mg0,48,net_load_params, 0.5, 0.5
```

```
Q3, rewards3 = training_Q_Learning(mg0,48,net_load_params, 0.9, 0.7
```

```
--- Training with Alpha=0.1, Epsilon=0.9, Gamma=0.99 ---
```

```
-----
|           WELCOME TO PYMGRID           |
-----
```

```
Run 1 Training Progress: Episode 300/300
```

```
--- Training with Alpha=0.5, Epsilon=0.5, Gamma=0.5 ---
```

```
-----
|           WELCOME TO PYMGRID           |
-----
```

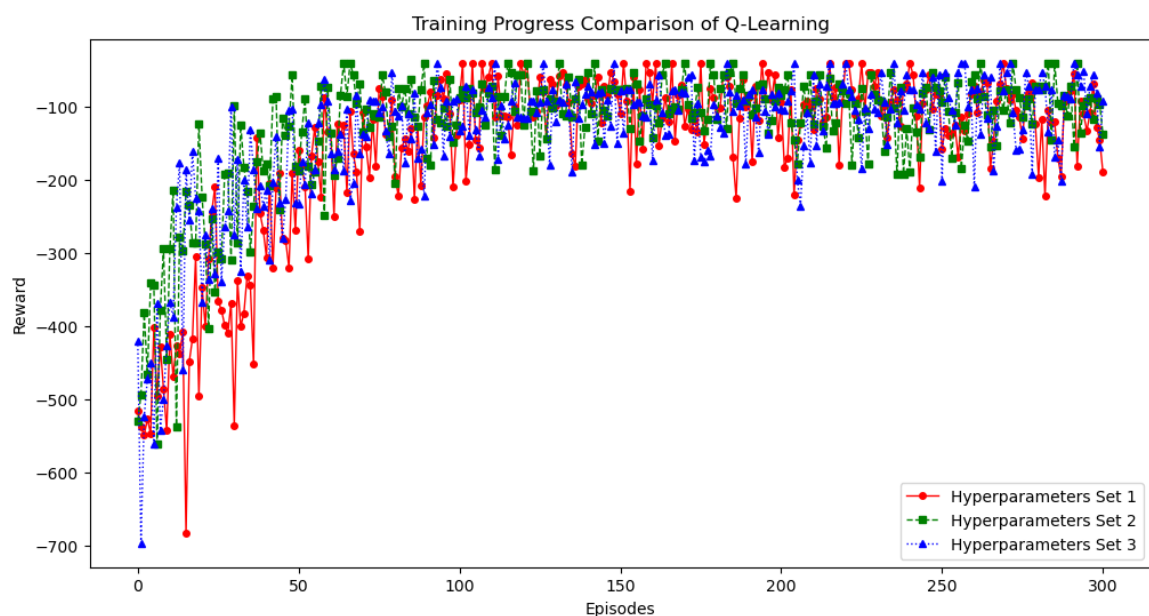
```
Run 2 Training Progress: Episode 300/300
```

```
--- Training with Alpha=0.9, Epsilon=0.7, Gamma=0.8 ---
```

```
-----
|           WELCOME TO PYMGRID           |
-----
```

```
Run 3 Training Progress: Episode 300/300
```

```
In [73]: # Plotting the training progress
# do NOT change
plt.figure(figsize=(12, 6))
plt.plot(rewards1, label='Hyperparameters Set 1', color='r', linestyle='--', marker='o')
plt.plot(rewards2, label='Hyperparameters Set 2', color='g', linestyle='--', marker='x')
plt.plot(rewards3, label='Hyperparameters Set 3', color='b', linestyle='--', marker='^')
plt.xlabel('Episodes')
plt.ylabel('Reward')
plt.title('Training Progress Comparison of Q-Learning')
plt.legend()
plt.show()
```



2. Testing Performance

```
In [66]: def testing_Q_Learning(mg, Q, horizon):
mg.reset(testing=True)
```

```

total_cost = 0
costs = [] # List to store single-step info
states = []
actions = []

for i in range(horizon):
    net_load = round(mg.load - mg.pv) # mg.load and mg.pv can be
    soc = round(mg.battery.soc, 1) # mg.battery.soc to get the
    s = (net_load, soc) # states combination of new states
    a = max_dict(Q[s])[0] # action with max value for the given

    if soc == 0.2 and a == 1: # do NOT change
        a = 2 # do NOT change
    if soc == 1 and a == 0: # do NOT change
        a = 1 # do NOT change

    actions.append(a) # Store the action
    action = actions_agent(mg, a)
    status = mg.run(action)
    cost = mg.get_cost()

    if a == 0: # do NOT change
        cost -= 35 # do NOT change

    total_cost += cost
    costs.append(total_cost)
    states.append((int(net_load), round(mg.battery.soc, 1))) #

    net_load = round(mg.load - mg.pv)
    soc = round(mg.battery.soc, 1)
    s_ = (net_load, soc)
    a_ = max_dict(Q[s_])[0]

    s, a = s_, a_

return costs, states, actions

```

In [77]: costs, states, actions = testing_Q_Learning(mg0, Q1, 48) # hint: us

```

In [76]: # Plotting the testing performance using your chosen set of hyperpa
# do Not Change
# do Not Change
plt.figure(figsize=(12, 6))
time_steps = range(len(costs))

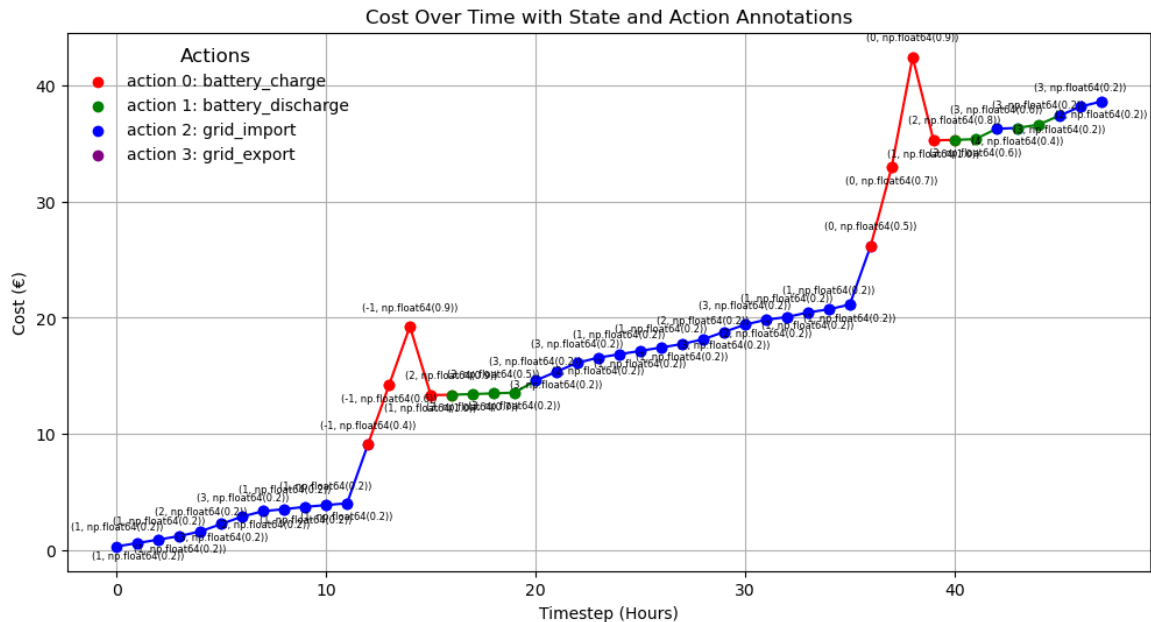
colors = {0: 'red', 1: 'green', 2: 'blue', 3: 'purple'}
actions_legend = ['battery_charge', 'battery_discharge', 'grid_imp

for i, (state, action) in enumerate(zip(states, actions)):
    plt.plot(time_steps[i:i+2], costs[i:i+2], color=colors[action],
    offset_y = 10 if i % 2 == 0 else -10
    plt.annotate(str(state), (time_steps[i], costs[i]), textcoords=
        xytext=(0, offset_y), ha='center', fontsize=6)
for action, color in colors.items():
    plt.scatter([], [], color=color, label=f"action {action}: {acti

```

```
plt.legend(loc='upper left', title='Actions', frameon=False, title_
plt.xlabel('Timestep (Hours)')
plt.ylabel('Cost (€)')
plt.title('Cost Over Time with State and Action Annotations')
plt.grid(True)
plt.show()

# Print the cost with two decimal places
total_cost = costs[-1]
print(f"Total cost is: {total_cost:.2f} €")
# do Not Change
# do Not Change
```



Total cost is: 38.64 €

3. Questions and discussion

Please write down your answers for the following 4 questions:

1. Base on above training figure, please explain how you choose the optimal set of hyperparameters for testing from the three sets.

Answer: All the three different sets of hyperparameters by episode 100, had come together, but the testing costs were different. The total cost for Run 1 ($\alpha=0.1$) and Run 3 ($\alpha=0.9$) was €38.64, but the total cost for Run 2 ($\alpha=0.5$) was €54.60, which was higher. Choose Q1 as the best option it has a low learning rate learning rate makes Q-value updates more stable. Large alpha values can make Q-values a bit unstable in future episodes, while very low alpha values always converge, even if it takes longer.

2. Please explain why SARSA is an on-policy learning and Q-learning is an off-policy learning

Answer: SARSA is an on-policy learning because it updates using the action

based on the next action taken based on the same policy,. It is called on-policy learning because the policy being evaluated and tested are the same. Equation: $Q[s][a] += \alpha(r + \gamma Q[s'][a'] - Q[s][a])$, where a' follows the same epsilon-greedy policy.

Q-learning updates the by checking for local maxima regardless of what action is taken. It evaluates the greedy policy while following exploratory policy, hence it is a off-policy learning. Equation: $Q[s][a] += \alpha * (r + \gamma * \max_{a'} Q[s][a'] - Q[s][a])$ updates using max $Q[s]$